Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 8ο : Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών

Όνομα: Σκαρμούτσος Σπυρίδων ΑΜ: 03121644
Github repo: https://github.com/Sskarm/advcomparch.git

Άσκηση 1η: Πρόβλεψη Εντολών Άλματος

Αυτή η άσκηση πρόκειται για την πειραματική αξιολόγηση διάφορων predictors που εφαρμόζονται πάνω σε benchmarks, όσων αφορά τα άλματα και τα branches. Τα benchmarks που μελετάμε είναι τα εξής:

1. 403.gcc 2. 410.bwaves 3. 416.gamess 4. 429.mcf
5. 433.milc 6. 435.gromacs 7. 436.cactusADM 8. 450.soplex
9. 459.GemsFDTD 10. 470.lbm 11. 483.xalancbmk

Για την αξιολόγηση της επίδοσης ενός predictor χρησιμοποιούμε τη μετρική direction Mispredictions Per Thousand Instructions (direction MPKI).

5.2 Ανάλυση εντολών άλματος

Χρησιμοποιούμε το cslab_branch_stats.cpp και για κάθε benchmark και βρέθηκε ο αριθμός των εντολών άλματος που εκτελέστηκαν και το ποσοστό αυτών που ανήκουν σε κάθε κατηγορία (conditional-taken, conditional-not taken κλπ.) τόσο για τα train όσο και για τα ref inputs.

Τα άλματα για τα train inputs:

Benchmark	Total Branches	Conditional Taken	Conditional Non-Taken	Unconditional Branches	Calls	Returns
403.gcc	754266405	269637287	304816996	66617406	56597360	56597356
410.bwaves	31873091519	21133243296	8406202329	1791546372	271049763	271049759
416.gamess	14348665729	6258962427	6983822129	863110726	121385227	121385220
429.mcf	3862678253	1332802279	2377832032	97772754	27135596	27135592
433.milc	2240721549	1242011774	541177932	21600372	217965739	217965732
435.gromacs	18177451950	5651208496	11410523474	778103288	168808348	168808344
436.cactusADM	170077085	142229935	18767160	1661514	3709240	3709236
450.soplex	1255127053	636674514	482534228	45598413	45159951	45159947
459.GemsFDTD	3418905867	2899775841	354670604	50413476	57022975	57022971
470.lbm	1356405014	176491991	922281969	252368700	2631179	2631175
483.xalancbmk	54611322567	8865640336	19089901678	5006932937	10824423810	10824423806

Τα ποσοστά επί τις εκατώ για την κάθε κατηγορία στα training inputs είναι:

Benchmark	Conditional Taken	Conditional Non-Taken	Unconditional Branches	Calls	Returns
403.gcc	35.75%	40.41%	8.83%	7.50%	7.50%
410.bwaves	66.30%	26.37%	5.62%	0.85%	0.85%
416.gamess	43.62%	48.67%	6.02%	0.85%	0.85%
429.mcf	34.50%	61.56%	2.53%	0.70%	0.70%
433.milc	55.43%	24.15%	0.96%	9.73%	9.73%
435.gromacs	31.09%	62.77%	4.28%	0.93%	0.93%
436.cactusADM	83.63%	11.03%	0.98%	2.18%	2.18%
450.soplex	50.73%	38.45%	3.63%	3.60%	3.60%
459.GemsFDTD	84.82%	10.37%	1.47%	1.67%	1.67%
470.lbm	13.01%	67.99%	18.61%	0.19%	0.19%
483.xalancbmk	16.23%	34.96%	9.17%	19.82%	19.82%

Τα άλματα για τα ref inputs:

Benchmark	Total Branches	Conditional Taken	Conditional Non-Taken	Unconditional Branches	Calls	Returns
403.gcc	33843770087	12296842130	13571514673	3134010276	2420701506	2420701502
410.bwaves	128568447429	88592483503	34014692949	3853748387	1053761297	1053761293
416.gamess	73894112363	34756621054	32132857316	5481924620	761354694	761354679
429.mcf	67874045798	25764447128	39087003883	2224162743	399216024	399216020
433.milc	67919982199	36675874303	16584639449	1322820264	6668324095	6668324088
435.gromacs	84256708162	26179756735	52980565854	3595805919	750289829	750289825
436.cactusADM	4364068636	4155679204	162691978	8434376	18631541	18631537
450.soplex	57466168415	31460495574	20383647486	1881871267	1870077046	1870077042
459.GemsFDTD	47124871693	33974307159	7769791232	2458154400	1461309453	1461309449
470.lbm	15278517633	527480282	11071876830	3673870659	2644933	2644929
483.xalancbmk	286243848086	75673001691	161638064234	7462347327	20735217419	20735217415

Τα ποσοστά επί τις εκατώ για την κάθε κατηγορία στα ref inputs είναι:

Benchmark	Conditional Taken	Conditional Non-Taken	Unconditional Branches	Calls	Returns
403.gcc	36.33%	40.10%	9.26%	9.26%	7.15%
410.bwaves	68.91%	26.46%	3.00%	3.00%	0.82%
416.gamess	47.04%	43.49%	7.42%	7.42%	1.03%
429.mcf	37.96%	57.59%	3.28%	3.28%	0.59%
433.milc	54.00%	24.42%	1.95%	1.95%	9.82%
435.gromacs	31.07%	62.88%	4.27%	4.27%	0.89%
436.cactusADM	95.22%	3.73%	0.19%	0.19%	0.43%
450.soplex	54.75%	35.47%	3.27%	3.27%	3.25%
459.GemsFDTD	72.09%	16.49%	5.22%	5.22%	3.10%
470.lbm	3.45%	72.47%	24.05%	24.05%	0.02%
483.xalancbmk	26.44%	56.47%	2.61%	2.61%	7.24%

5.3 N-bit predictors

i) Η υλοποίηση των n-bit predictor γίνονται δημιουργώντας στο cslab_branch.cpp 4 καινούριους NbitPredictor με ορίσματα (14,1) , (14,2), (14,3) , (14,4) αρα 2^14 BHT entries και 1 εως 4 bit αναλόγως. Οι αποδόσεις φαίνονται παρακάτω:

Benchmark	1-bit Correct	1-bit Incorrect	2-bit Correct	2-bit Incorrect	Total Instructions
403.gcc	502949981	71501389	520133471	54317899	3184170871
410.bwaves	22070332585	7469113041	25413369985	4126075641	300165002696
416.gamess	11679144127	1563640429	12323343519	919441037	191137079537
429.mcf	3371435895	339198416	3469506200	241128111	18199704182
433.milc	789062174	994127111	1285315772	497873513	38323372296
435.gromacs	15341735683	1719996296	15712719927	1349012052	445004175612
436.cactusADM	155195552	5801612	157850699	3146465	80776878079
450.soplex	1020337252	102695485	1059334596	63698141	7439861708
459.GemsFDTD	3168059319	86387115	3206879631	47566803	108776742176
470.lbm	1080176096	18597864	1066594479	32179481	96483398089
483.xalancbmk	24409279555	3546262452	25444084024	2511457983	224117738434

Benchmark	3-bit Correct	3-bit Incorrect	4-bit Correct	4-bit Incorrect	Total Instructions
403.gcc	522936157	51515213	522682389	51768981	3184170871
410.bwaves	25456986638	4082458988	25458191257	4081254369	300165002696
416.gamess	12277113539	965671017	12216393397	1026391159	191137079537
429.mcf	3476616030	234018281	3475322089	235312222	18199704182
433.milc	1285473028	497716257	1285501048	497688237	38323372296
435.gromacs	15582286052	1479445927	15554875849	1506856130	445004175612
436.cactusADM	157947092	3050072	157954612	3042552	80776878079
450.soplex	1058330620	64702117	1056307265	66725472	7439861708
459.GemsFDTD	3208487938	45958496	3208739123	45707311	108776742176
470.lbm	1051686086	47087874	1023615097	75158863	96483398089
483.xalancbmk	25647917382	2307624625	25656751774	2298790233	224117738434

(Τα total instructions είναι κοινά στα δυο πινακάκια και είναι εκεί για reference).

Στον παρακάτω πίνακα φαίνεται η μετρική MPKI (Mispredictions per Thousand Instructions).

Benchmark	MPKI 1 bit	MPKI 2 bit	MPKI 3 bit	MPKI 4 bit
403.gcc	22.46	17.06	16.18	16.26
410.bwaves	24.88	13.75	13.60	13.60
416.gamess	8.18	4.81	5.05	5.37
429.mcf	18.64	13.25	12.86	12.93
433.milc	25.94	12.99	12.99	12.99
435.gromacs	3.87	3.03	3.32	3.39
436.cactusADM	0.07	0.04	0.04	0.04
450.soplex	13.80	8.56	8.70	8.97
459.GemsFDTD	0.79	0.44	0.42	0.42
470.lbm	0.19	0.33	0.49	0.78
483.xalancbmk	15.82	11.21	10.30	10.26
AVERAGE	12.24	7.77	7.63	7.73

Από την παραπάνω μετρική βλέπω ότι δεν υπάρχει significant βελτίωση στα train inputs κατά την αύξηση των bit οπότε θα διάλεγα τον Nbit-16K-2bit καθώς έχει και πολύ χαμηλό MPKI αλλά και θα είναι πιο γρήγορος λόγω μικρότερου κυκλωματικού complexity και θα απαιτεί λιγότερα resources (λογικές πύλες και καλώδια) στην κατασκευή του.

ii) Για N=2 υλοποίησα την κλάση FSM2BitPredictor

```
VOID InitPredictors()
{

    // // Pentium-M predictor
    // PentiumMBranchPredictor *pentiumPredictor = new PentiumMBranchPredictor();
    // branch_predictors.push_back(pentiumPredictor);

    branch_predictors.push_back(new FSM2BitPredictor(14, "ABACBDCD", 3, "FSM-1"));
    branch_predictors.push_back(new FSM2BitPredictor(14, "ABACADCD", 3, "FSM-2"));
    branch_predictors.push_back(new FSM2BitPredictor(14, "ABADBDCD", 3, "FSM-3"));
    branch_predictors.push_back(new FSM2BitPredictor(14, "ABADADCD", 3, "FSM-4"));
    branch_predictors.push_back(new FSM2BitPredictor(14, "ABADBDCC", 3, "FSM-5"));

    /* ... */
}
```

Κάνει extend την κλάση BranchPredictor και δέχεται σαν όρισμα τα bits του BHT, την συμβολοσειρά από A,,B,C,D που ορίζει σύμφωνα με το paper Optimal 2-bit branch predictors το κατάλληλο FSM.(πρακτικά για κάθε "node" υπάρχει ένα touple T,NT που δείχνει που πάνε με το κατάλληλο input). Με αυτόν τον τρόπο φαίνεται η κωδικοποίηση των 5 FSM εκ των οποίων το 1° είναι ο 2bit counter η αλλιώς Saturating Counter Predictor. Τα στατιστικά για τους predictor είναι τα εξής:

Benchmark	FSM-1 Correct	FSM-1 Incorrect	FSM-2 Correct	FSM-2 Incorrect	FSM-3 Correct	FSM-3 Incorrect
403.gcc	520234296	54314120	519569952	54978464	519923178	54625238
410.bwaves	25413369983	4126075642	25206009332	4333436293	25408377095	4131068530
416.gamess	12323343663	919440887	12118013233	1124771317	12265409218	977375332
429.mcf	3469506200	241128111	3469570531	241063780	3470069576	240564735
433.milc	1285315772	497873513	435660251	1347529034	1285220947	497968338
435.gromacs	15712719877	1349012009	15326550287	1735181599	15692863138	1368868748
436.cactusADM	157850651	3146467	157985923	3011195	157983082	3014036
450.soplex	1059162631	62546470	1058369501	63339600	1058801702	62907399
459.GemsFDTD	3206879608	47566826	3206629176	47817258	3206097646	48348788
470.lbm	1066594479	32179481	1066313845	32460115	1063399579	35374381
483.xalancbmk	25444084029	2511457985	25375611298	2579930716	25144068601	2811473413

Benchmark	FSM-4 Correct	FSM-4 Incorrect	FSM-5 Correct	FSM-5 Incorrect	Total Instructions
403.gcc	519413461	55134955	516758833	57789583	3184507251
410.bwaves	25386037093	4153408532	23654181564	5885264061	300165002694
416.gamess	12315933772	926850778	12086655922	1156128628	191137079600
429.mcf	3468751383	241882928	3459198116	251436195	18199704182
433.milc	1285201211	497988074	793083317	990105968	38323372296
435.gromacs	15685903987	1375827899	15464089965	1597641921	445004175949
436.cactusADM	157983806	3013312	157964311	3032807	80776877872
450.soplex	1058252687	63456414	1045768503	75940598	7440656697
459.GemsFDTD	3206641447	47804987	3192537320	61909114	108776742205
470.lbm	1063036344	35737616	1067814001	30959959	96483398089
483.xalancbmk	25261601907	2693940107	24999057134	2956484880	224117738457

Τα παραπάνω στατιστικά συνοψίζονται με την χρήση του metric MPKI όπως αναλύθηκε πριν.

Benchmark	MPKI FSM 1	MPKI FSM 2	MPKI FSM 3	MPKI FSM 4	MPKI FSM 5
403.gcc	17.055738	17.264355	17.153435	17.313496	18.147104
410.bwaves	13.746025	14.436847	13.762659	13.837085	19.606763
416.gamess	4.8103743	5.8846317	5.1134784	4.8491417	6.0486884
429.mcf	13.249013	13.245478	13.218057	13.290487	13.8154
433.milc	12.991381	35.162068	12.993855	12.99437	25.835565
435.gromacs	3.0314592	3.8992479	3.0760807	3.0917191	3.5901729
436.cactusADM	0.0389526	0.0372779	0.0373131	0.0373041	0.0375455
450.soplex	8.4060416	8.5126357	8.4545493	8.5283351	10.206169
459.GemsFDTD	0.4372886	0.4395908	0.4444773	0.439478	0.5691393
470.lbm	0.3335235	0.3364321	0.366637	0.3704017	0.3208838
483.xalancbmk	11.205976	11.511497	12.544627	12.0202	13.191659
AVERAGE	7.7550701	10.066369	7.9241063	7.8883653	10.124463

Βλέποντας τα παραπάνω average MPKI καταλαβαίνει κανείς ότι αν δεν έχει κάποιο συγκεκριμένο use case και θέλει να έχει κάποια γενική χρήση που θεωρούμε ότι καλύπτεται από τα benchmarks που καλύπτει αυτή η εργασία τότε θα επέλεγε το 1° FSM (2 bit counter) καθώς κάνει καλύτερο perform σε σχέση με τα άλλα FSMs (μικρότερο MPKI => καλύτερη απόδοση).

Rank	State diagram for machine
1.	
2.	
3.	
4.	
5.	

Αριστερά φαίνονται και τα FSMs

1.ABACBDCD

2.ABACADCD

3.ABADBDCD

4.ABADADCD

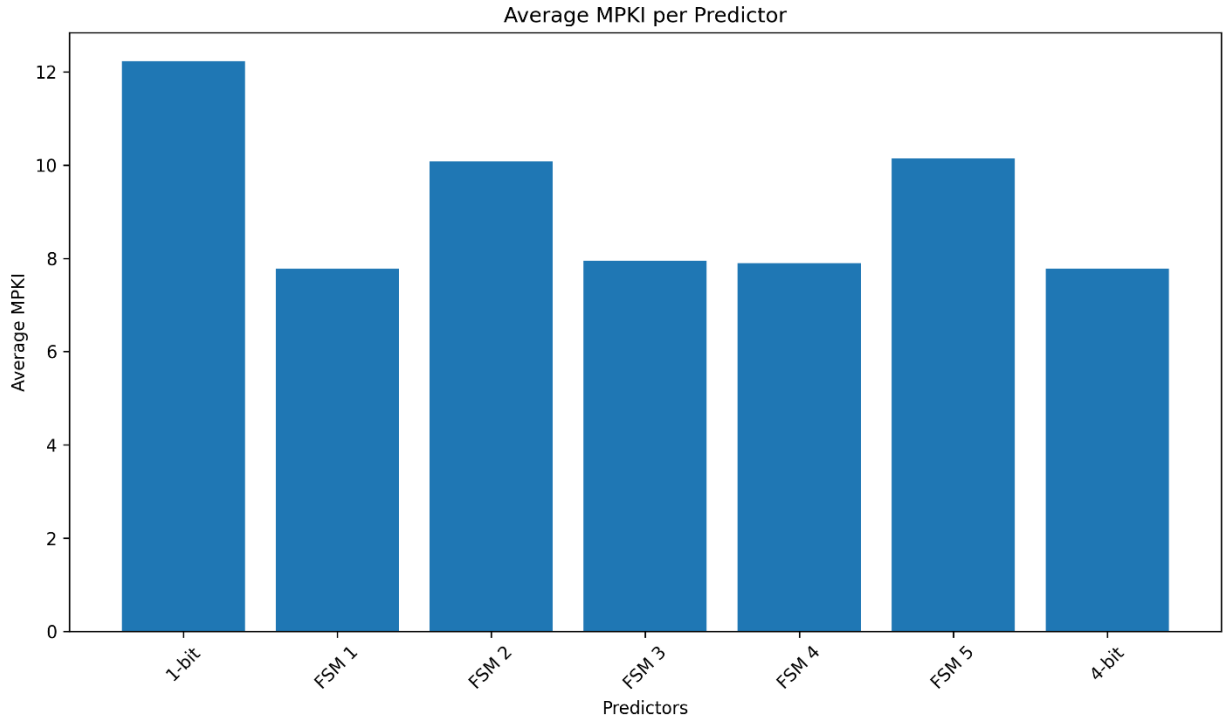
5.ABADBDCC

```cpp
class FSM2BitPredictor : public BranchPredictor{
    public:
        FSM2BitPredictor(unsigned index_bits_,const std::string& transitions_,unsigned output_mask_,const std::string& name_)
        : BranchPredictor(),index_bits(index_bits_),transitions(transitions_),output_mask(output_mask_),name(name_){
            table_entries = 1 << index_bits;
            STATE = new unsigned char[table_entries];

            for (unsigned i = 0; i < table_entries; i++)
                STATE[i] = 0;   // αρχική κατάσταση A

            // transitions string: A0 A1 B0 B1 C0 C1 D0 D1
            for (int s = 0; s < 4; s++) {
                next_state[s][0] = charToState(transitions[2 * s]);
                next_state[s][1] = charToState(transitions[2 * s + 1]);
            }

            // output bits για A,B,C,D
            // π.χ. output_mask = 3 => binary 0011 => A=0, B=0, C=1, D=1
            for (int s = 0; s < 4; s++) {
                prediction_bit[s] = (output_mask >> (3 - s)) & 1;
            }
        }

        ~FSM2BitPredictor() {
            delete[] STATE;
        }

        virtual bool predict(ADDRINT ip, ADDRINT target) {
            unsigned idx = ip % table_entries;
            unsigned state = STATE[idx];
            return prediction_bit[state];
        }

        virtual void update(bool predicted, bool actual, ADDRINT ip, ADDRINT target) {
            unsigned idx = ip % table_entries;
            unsigned curr = STATE[idx];
            STATE[idx] = next_state[curr][actual ? 1 : 0];
            updateCounters(predicted, actual);
        }

        virtual string getName() {
            std::ostringstream stream;
            stream << name << " (" << transitions << ":" << output_mask << ")";
            return stream.str();
        }

    private:
        unsigned char charToState(char c) {
            if (c == 'A') return 0;
            if (c == 'B') return 1;
            if (c == 'C') return 2;
            return 3; // D
        }

        unsigned index_bits;
        unsigned table_entries;
        unsigned char* STATE;

        std::string transitions;
        unsigned output_mask;
        std::string name;

        unsigned char next_state[4][2];
        bool prediction_bit[4];
};
```

5.3

iii) Για λόγους ευκρίνειας από εδώ και πέρα θα παρουσιάζω μόνο τα MPKI για τους predictors παρόλα αυτά στο repo στο location advcomparch-ex1-helpcode/csvs/32K_predictors.csv υπάρχουν και τα ακριβή νούμερα για τα correct και incorrect predictions.

Benchmark	MPKI 1 bit	MPKI FSM 1	MPKI FSM 2	MPKI FSM 3	MPKI FSM 4	MPKI FSM 5	MPKI 4 bit
403.gcc	22.31711	17.06008156	17.268342	17.159862	17.31789	18.151188	16.57297
410.bwaves	24.88336	13.74602516	14.436847	13.762659	13.837085	19.606763	13.59671
416.gamess	8.180192	4.810376222	5.8846341	5.1134805	4.8491437	6.0486909	5.371111
429.mcf	18.63758	13.24901265	13.245478	13.218057	13.290487	13.8154	12.92945
433.milc	25.94049	12.99138051	35.162068	12.993855	12.99437	25.835565	12.98655
435.gromacs	3.863887	3.031460432	3.8992495	3.0760819	3.0917203	3.5901743	3.387616
436.cactusADM	0.071817	0.03895257	0.0372779	0.0373131	0.0373041	0.0375455	0.037724
450.soplex	13.79258	8.551953203	8.6535416	8.6183861	8.6725332	10.374136	9.012606
459.GemsFDTD	0.794057	0.43728757	0.4395898	0.4444762	0.439477	0.5691379	0.421004
470.lbm	0.192757	0.333523504	0.3364321	0.366637	0.3704017	0.3208838	0.778982
483.xalancbmk	15.83363	11.20596289	11.511484	12.544612	12.020186	13.191644	10.4997
AVERAGE	12.22795	7.768728753	10.07954	7.9395837	7.9018725	10.140103	7.781311



Παρατηρώ από τους παραπάνω plots ότι ο 1bit 32K predictor είναι ο χειρότερος καθώς με 1 bit είναι πολύ απλός για να είναι αποτελεσματικός σε μεγάλο πλήθος benchmarks (με ποικιλία complexity). Από εκεί και μετά παρατηρούμε 2 ομάδες μια είναι οι 2-bit fsm 1,3,4 και ο 4-bit (8k bht) που είναι οι "αποτελεσματική" ομάδα και οι fsm 2 και 5 που είναι η πιο κακή ομάδα. Για τους 2 και 5 το περιμέναμε καθώς από τους 5 πειραματικά καλύτερους predictors ήταν αυτοι που και στο προηγούμενο ερώτημα είχαν χειρότερη απόδοση. Τέλος μιας και το hardware παραμένει το ίδιο η απόφαση είναι απλή θα επέλεγα τον predictor με το χαμηλότερο MPKI δηλαδή το FSM-1 η αλλιώς τον απλό saturating counter. (Σημείωση παρατηρούμε ότι με την αύξηση του complexity από 2 σε 4-bit όχι μόνο δεν καλυτερεύει η απόδοση του predictor αλλά γίνεται και χειρότερη οπότε δεν χρειάζεται να διαλέξουμε τον πιο complex predictor.)

5.4 Μελέτη του BTB

Για την αξιολόγηση του BTB χρησιμοποιήσαμε ως βασική μετρική το direction MPKI, δηλαδή τον αριθμό λανθασμένων direction προβλέψεων ανά 1000 εντολές. Επιπλέον, υπολογίσαμε και τον αριθμό σωστών target προβλέψεων ανά 1000 εντολές, χρησιμοποιώντας το πεδίο TargetCorrect, ώστε να έχουμε και μια εικόνα της αποτελεσματικότητας του BTB ως μηχανισμού πρόβλεψης στόχου. Η δεύτερη αυτή μετρική δεν αποτελεί target misprediction rate, αλλά χρησιμεύει συγκριτικά μεταξύ διαφορετικών BTB οργανώσεων.

benchmark	MPKI BTB-512-1	MPKI BTB-512-2	MPKI BTB-256-2	MPKI BTB-256-4	MPKI BTB-128-2	MPKI BTB-128-4	MPKI BTB-64-4	MPKI BTB-64-8
403.gcc	46.286202	45.66113095	44.10753	43.91748	44.80413643	44.57812	45.85785	45.54727218
410.bwaves	22.822863	22.74169913	22.5051	22.72639	22.62788022	22.09414	21.93703	21.70957285
416.gamess	12.877508	16.04930339	12.84201	13.0744	9.089374946	9.502099	8.534282	8.406164887
429.mcf	125.77529	126.6362927	125.8039	125.6945	123.0449026	123.4767	122.7722	117.0711111
433.milc	13.980911	13.9791869	13.97894	13.97913	13.97479971	13.80093	13.65359	13.9734075
435.gromacs	7.1140113	7.099907846	6.925963	6.838212	6.389021782	6.709179	6.263495	6.471345727
436.cactusADM	0.0760545	0.075775261	0.074718	0.075333	0.074753347	0.0742	0.074723	0.073916734
450.soplex	33.046937	33.03342213	31.43966	31.7869	23.31335708	23.06263	23.61027	22.17888363
459.GemsFDTD	1.4078355	1.384376991	1.455735	1.3952	1.519802458	1.451081	1.561008	1.578491547
470.lbm	6.7764253	6.786279836	6.78628	6.786281	6.732389508	6.732387	6.707759	6.707760048
483.xalancbmk	39.152971	36.1607737	36.26182	35.26433	39.35220508	37.23864	44.22088	45.79110473
AVERAGE	28.119728	28.14619534	27.47106	27.41256	26.4475112	26.24728	26.83574	26.31900282

Παρακάτω παραθέτω τον πίνακα με το Target Correct per 1000 Instructions. Τον ονομάζω TCPKI και δεν είναι κάποιος well known δείκτης αλλά μπορεί να δώσει μια καλή ιδέα χωρίς να διαστρεβλώνει τα δεδομένα εφόσον δεν έχουμε τα stats για το taken direction hit.

benchmark	TCPKI BTB-512-1	TCPKI BTB-512-2	TCPKI BTB-256-2	TCPKI BTB-256-4	TCPKI BTB-128-2	TCPKI BTB-128-4	TCPKI BTB-64-4	TCPKI BTB-64-8
403.gcc	95.33883093	98.11862	93.742907	95.2160303	87.93297	89.64134	82.86105044	83.77749
410.bwaves	75.98078041	76.34824	76.213546	76.34000519	75.60699	76.02064	75.32142352	75.5065
416.gamess	36.67280555	36.89397	36.736688	36.85953088	36.37185	36.42059	35.81924818	35.85878
429.mcf	78.14507334	78.58229	78.456616	78.53450533	78.13192	78.30395	77.77835095	77.92629
433.milc	32.97176572	32.97224	32.971995	32.97216514	32.97127	32.97161	32.97040519	32.97054
435.gromacs	14.41055958	14.4436	14.431586	14.44354753	14.3109	14.37609	14.16771649	14.21167
436.cactusADM	1.78048731	1.78077	1.7803739	1.780474807	1.779267	1.779933	1.778575253	1.779213
450.soplex	91.28249593	91.66395	91.072781	91.66950511	89.9686	90.58494	88.92456702	88.85087
459.GemsFDTD	26.95646305	27.01486	26.904486	26.96668737	26.74339	26.78993	26.62726608	26.60231
470.lbm	4.444905388	4.444908	4.4449038	4.444905481	4.444894	4.444898	4.444884348	4.444885
483.xalancbmk	53.27830374	56.11742	52.349841	53.39788651	43.81986	46.79371	34.21883525	34.62628
AVERAGE	46.47840645	47.12553	46.282339	46.60229488	44.73472	45.28433	43.17384752	43.32317

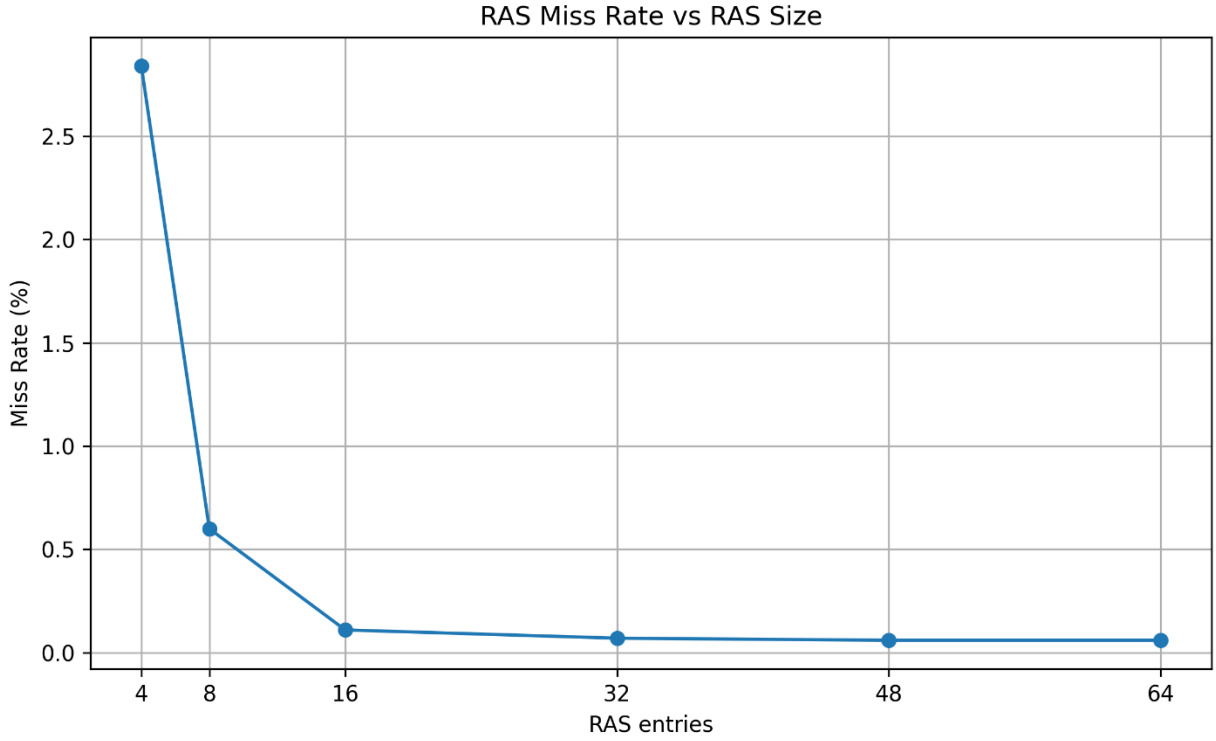
Με βάση τα αποτελέσματα, η οργάνωση BTB-128-4 αποτελεί την καλύτερη επιλογή. Παρουσιάζει το μικρότερο average direction MPKI (26.247), άρα τη χαμηλότερη συχνότητα direction mispredictions ανά 1000 εντολές. Παρότι δεν μεγιστοποιεί το πλήθος σωστών target predictions ανά 1000 εντολές, διατηρεί αρκετά καλή επίδοση και σε αυτή τη μετρική (45.284), προσφέροντας την καλύτερη συνολική ισορροπία μεταξύ direction accuracy και target prediction effectiveness. Οι μεγαλύτερες οργανώσεις (512-*) πετυχαίνουν περισσότερα σωστά targets ανά 1000 εντολές, αλλά με σημαντικά χειρότερο MPKI, ενώ οι μικρότερες (64-*) έχουν ανταγωνιστικό MPKI αλλά σαφώς χειρότερη επίδοση στα σωστά targets. Επομένως, το BTB-128-4 είναι η βέλτιστη trade-off επιλογή.

Τα csv με όλες τις μετρικές βρίσκονται στο repo στο εξής αρχείο: advcomparch-ex1-helpcode/csvs/btb_predicts.csv

5.5 Μελέτη του RAS

Για την μελέτη του RAS υπολογίζουμε το ποσοστό αστοχίας, δηλαδη incorrect*100/(correct+incorrect) για να αποφανθούμε ποιό είναι το κατάλληλο μέγεθος για την RAS.

benchmark	RAS-4	RAS-8	RAS-16	RAS-32	RAS-48	RAS-64
403.gcc	4.13%	0.30%	0.02%	0.00%	0.00%	0.00%
410.bwaves	0.96%	0.00%	0.00%	0.00%	0.00%	0.00%
416.gamess	0.72%	0.00%	0.00%	0.00%	0.00%	0.00%
429.mcf	4.75%	0.53%	0.00%	0.00%	0.00%	0.00%
433.milc	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
435.gromacs	1.04%	0.00%	0.00%	0.00%	0.00%	0.00%
436.cactusADM	0.82%	0.06%	0.00%	0.00%	0.00%	0.00%
450.soplex	0.01%	0.00%	0.00%	0.00%	0.00%	0.00%
459.GemsFDTD	2.89%	0.00%	0.00%	0.00%	0.00%	0.00%
470.lbm	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
483.xalancbmk	15.96%	5.67%	1.23%	0.77%	0.69%	0.61%
AVERAGE	2.84%	0.60%	0.11%	0.07%	0.06%	0.06%



Προφανώς αναζητούμε το «γόνατο» του Plot. Οι τιμές από 16 εώς 64 είναι αποδεκτές αλλά θεωρώ ότι θα διάλεγα τον RAS με μέγεθος 48 εγγραφών καθώς φαίνεται να είναι ο καλυτερος σε ισοβαθμία με τον 64^{ων} εγγραφών αλλά είναι και μικρότερος άρα πιο απλός-γρήγορος.

Τα csv με τις μετρήσεις βρίσκονται στο repo στο εξης location: advcomparch-ex1-helpcode/csvs/ras_stats.csv

5.6 Perceptrons

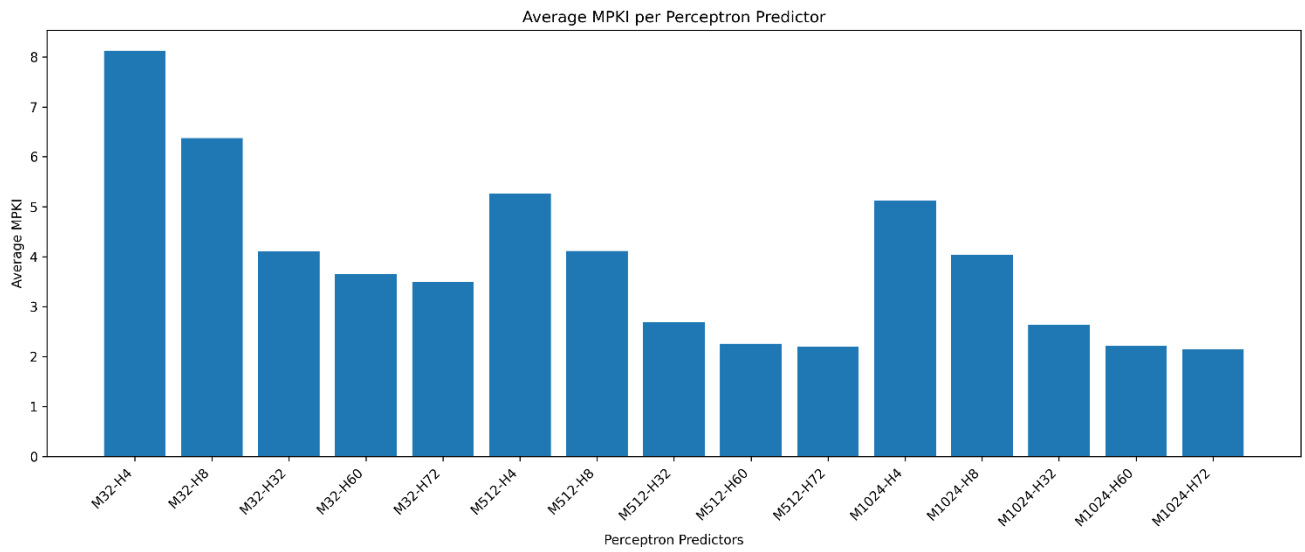
Το metric είναι το γνωστό MPKI. Τα πληρη metric βρισκονται στο repo advcomparch-ex1-helpcode/csvs/perceptron_stats.csv

Benchmark	M32-H4 MKPI	M32-H8 MKPI	M32-H32 MKPI	M32-H60 MKPI	M32-H72 MKPI	M512-H4 MKPI	M512-H8 MKPI	M512-H32 MKPI
403.gcc	25.486331	20.96872365	15.184551	13.90481894	13.7037	12.97858165	9.68642	5.996691187
410.bwaves	13.139233	10.81123651	5.9748281	4.03363766	2.927578	11.23643117	9.237895	4.582572201
416.gamess	5.2939348	4.012549748	2.507566	2.050265226	2.006973	3.649536663	2.780426	1.60087031
429.mcf	10.841779	9.556600164	8.9481134	8.83671643	8.709124	9.85674279	8.925936	8.478246153
433.milc	3.3440301	1.521832566	0.0344342	0.028785541	0.028234	3.039323446	1.501678	0.033988162
435.gromacs	2.8009729	2.186743153	1.8651885	1.84016445	1.838357	2.10186594	1.735532	1.495036147
436.cactusADM	0.0364552	0.034998468	0.0244053	0.023820505	0.02369	0.033753384	0.032569	0.022800324
450.soplex	9.6625845	7.835392849	6.1187165	6.027111981	6.016165	7.884292786	6.926333	5.749991141
459.GemsFDTD	0.648899	0.307450576	0.1383532	0.134561963	0.132634	0.379288662	0.164337	0.136250422
470.lbm	0.1852011	0.183659037	0.1815358	0.180147376	0.180234	0.185393802	0.183674	0.181520628
483.xalancbmk	17.950281	12.72265767	4.220398	3.118859261	2.931368	6.594072394	4.074438	1.250169911
AVERAGE	8.1263365	6.376531308	4.1089173	3.652626303	3.499823	5.267207517	4.113567	2.684376053

Benchmark	M512-H60 MKPI	M512-H72 MKPI	M1024-H4 MKPI	M1024-H8 MKPI	M1024-H32 MKPI	M1024-H60 MKPI	M1024-H72 MKPI
403.gcc	5.347853	5.277815106	12.10693	9.096718898	5.550508	4.872952753	4.78762
410.bwaves	1.52687	1.087498943	11.09356	9.212217845	4.579534	1.525896593	1.086529
416.gamess	1.152889	1.108873895	3.597502	2.75305275	1.583173	1.13439562	1.089745
429.mcf	8.357598	8.227196798	9.763207	8.910973901	8.476794	8.355989331	8.225785
433.milc	0.028293	0.027731902	3.035405	1.497322614	0.033963	0.028271781	0.027701
435.gromacs	1.475433	1.475319	2.091287	1.733933312	1.492915	1.473075169	1.472781
436.cactusADM	0.022496	0.022440184	0.033343	0.032468475	0.022752	0.022450125	0.022389
450.soplex	5.672918	5.658880643	7.853453	6.919923841	5.740514	5.661208411	5.646063
459.GemsFDTD	0.133048	0.131358613	0.372319	0.162157771	0.136203	0.133679174	0.131399
470.lbm	0.180125	0.180221296	0.185393	0.183673216	0.18152	0.180123869	0.180221
483.xalancbmk	0.973915	0.923610507	6.278648	3.932493314	1.238999	0.973576458	0.921793
AVERAGE	2.26104	2.192813353	5.128277	4.039539631	2.639716	2.214692662	2.14473

Θα επέλεγα τον Perceptron predictor με M=1024 και n=72.

Ο λόγος είναι ότι από τα αποτελέσματά σου αυτός έχει το μικρότερο average MPKI, ίσο με 2.1447, άρα είναι ο πιο ακριβής predictor από όλους τους συνδυασμούς που δοκίμασες. Αφού στο 5.6 η βασική μετρική σύγκρισης είναι το direction MPKI, η φυσική επιλογή είναι ο predictor με τη χαμηλότερη τιμή.



5.6 Σύγκριση διαφορετικών predictors

Παρακατω φαινονται οι μετρικες MPKI για όλους τους predictors ολοκληρο το csv βρισκεται στο repo advcomparch-ex1-helpcode/csvs/all_predictors.csv

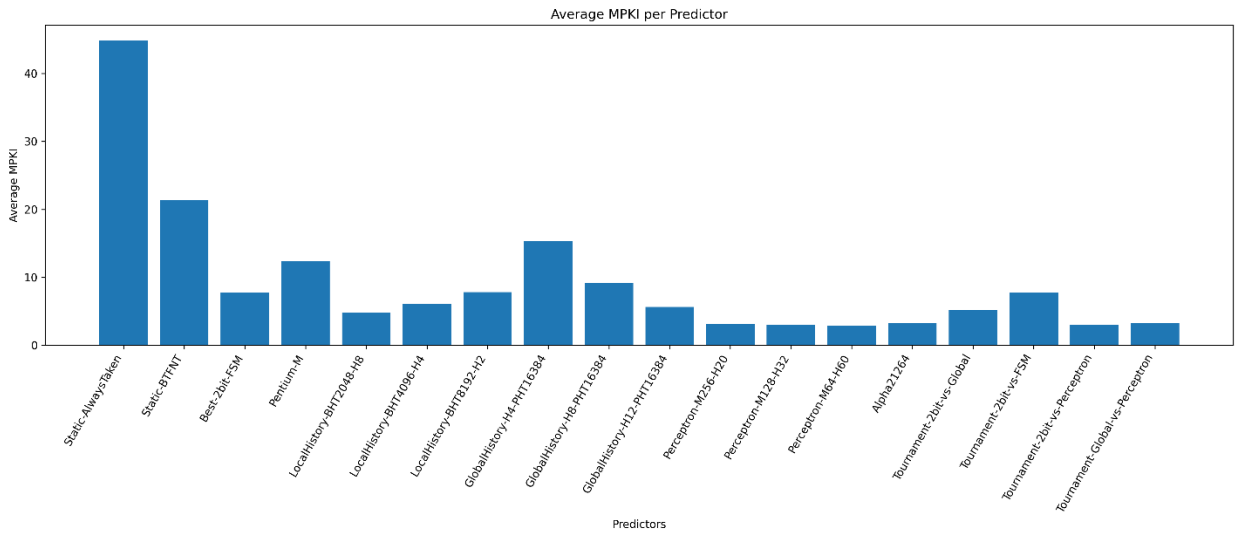
Benchmark	Static-AlwaysTaken MPKI	Static-BTFNT MPKI	Best-2bit-FSM (ABACBDCD:3) MPKI	Pentium-M MPKI	LocalHistory-BHT2048-H8 MPKI	LocalHistory-BHT4096-H4 MPKI
403.gcc	95.72972	54.51772	17.05609	32.26621	15.40436	17.94662
410.bwaves	28.00527	17.00081	13.74603	15.49432	7.563413	11.26505
416.gamess	36.53829	22.83728	4.810374	5.885299	1.670692	2.805944
429.mcf	130.6522	58.58697	13.24901	54.71854	11.01406	12.00615
433.milc	14.12135	13.38693	12.99138	1.282982	0.033926	0.063481
435.gromacs	25.64139	7.484964	3.031459	2.920656	2.163923	2.495744
436.cactusADM	0.232334	0.07008	0.038952	0.056408	0.024954	0.032083
450.soplex	64.90622	29.47315	8.377252	12.75814	7.022904	8.227406
459.GemsFDTD	3.260537	1.871671	0.437289	1.494721	0.237243	0.461146
470.lbm	9.558971	2.740991	0.333524	0.29271	0.180234	0.181255
483.xalancbmk	85.178	27.23747	11.20598	8.260746	7.761585	11.35366
AVERAGE	44.89312	21.38255	7.752485	12.31188	4.825209	6.076231

Benchmark	LocalHistory-BHT8192-H2 MPKI	GlobalHistory-H4-PHT16384 MPKI	GlobalHistory-H8-PHT16384 MPKI	GlobalHistory-H12-PHT16384 MPKI	Perceptron-M256-H20 MPKI	Perceptron-M128-H32 MPKI
403.gcc	19.86951	47.3245	31.12975	16.47585	7.77512	8.530613
410.bwaves	16.05539	20.6155	11.88097	8.23948	6.272443	4.5624
416.gamess	5.142602	16.4076	5.482657	2.810466	1.918172	1.802671
429.mcf	13.40879	16.20311	12.07882	9.817727	8.57993	8.527027
433.milc	3.088229	4.309432	2.456344	0.977958	0.035116	0.034092
435.gromacs	3.650476	6.081844	4.414786	2.002677	1.508019	1.53551
436.cactusADM	0.03929	0.065834	0.03861	0.026336	0.023638	0.023158
450.soplex	8.895408	21.48796	9.991721	7.584979	6.091731	5.84387
459.GemsFDTD	0.498351	1.093917	0.554652	0.227063	0.142462	0.136557
470.lbm	0.209168	0.243109	0.192025	0.180246	0.181628	0.181527
483.xalancbmk	14.8509	35.02162	22.73368	13.43558	2.038782	1.576203
AVERAGE	7.791645	15.3504	9.177637	5.616215	3.142458	2.977603

Benchmark	Perceptron-M64-H60 MPKI	Alpha21264 MPKI	Tournament-2bit-vs-Global MPKI	Tournament-2bit-vs-FSM MPKI	Tournament-2bit-vs-Perceptron MPKI	Tournament-Global-vs-Perceptron MPKI
403.gcc	10.17144	9.416159	13.75183	16.98622	8.556328	10.93459
410.bwaves	1.97387	4.895542	8.5141	13.72608	3.559882	3.509899
416.gamess	1.623546	0.981593	3.019657	4.800145	1.582805	1.525154
429.mcf	8.498022	8.453654	10.07147	13.03933	8.756385	8.680883
433.milc	0.02854	0.020822	2.226501	12.99145	0.028829	0.028433
435.gromacs	1.618536	1.498276	2.170534	3.020154	1.683947	1.705453
436.cactusADM	0.023121	0.023596	0.03318	0.037248	0.022999	0.023346
450.soplex	5.87437	6.078155	7.532631	8.397047	6.059709	5.983127
459.GemsFDTD	0.13365	0.15054	0.217445	0.434654	0.133088	0.133005
470.lbm	0.180139	0.177953	0.182462	0.33332	0.180301	0.178358
483.xalancbmk	1.705413	3.569351	9.175623	11.31716	2.038778	2.534776
AVERAGE	2.893695	3.205967	5.172312	7.7348	2.963914	3.203366

Με βάση τα αποτελέσματα της σύγκρισης, θα επέλεγα τον predictor Perceptron-M64-H60, καθώς παρουσίασε το μικρότερο average MPKI (2.8937) από όλους τους predictors που προσομοιώθηκαν. Αυτό σημαίνει ότι είχε τη χαμηλότερη συχνότητα λανθασμένων direction predictions ανά 1000 εντολές και συνεπώς την καλύτερη συνολική επίδοση. Παρότι κάποιοι hybrid και άλλοι perceptron predictors είχαν επίσης πολύ καλή συμπεριφορά, κανένας δεν ξεπέρασε τον συγκεκριμένο predictor. Επομένως, με κριτήριο καθαρά την prediction accuracy, ο Perceptron-M64-H60 αποτελεί τη βέλτιστη επιλογή.

Παρακατω φαίνεται το διάγραμμα με τα MPKI.



5.7 Ακρίβεια προσομοιώσεων

Benchmark	Static-AlwaysTaken MPKI	Perceptron-M128-H32 MPKI	Perceptron-M64-H60 MPKI	Tournament-2bit-vs-Perceptron MPKI
403.gcc	95.46375	6.54305	7.673041	6.693194
410.bwaves	15.59568	1.253539	0.568497	0.995216
416.gamess	30.81587	1.551483	1.487871	1.469688
429.mcf	128.5271	11.41802	11.47612	11.8361
433.milc	14.18621	0.022106	0.017644	0.017827
435.gromacs	25.58229	1.532093	1.615181	1.679469
436.cactusADM	0.059365	0.012801	0.012749	0.012783
450.soplex	58.1349	4.639467	4.573817	4.71359
459.GemsFDTD	5.263505	0.096013	0.095818	0.095502
470.lbm	8.691604	0.041563	0.039955	0.039943
483.xalancbmk	175.7598	1.480611	1.561153	1.476328
AVERAGE	50.73455	2.599159	2.647441	2.639058

Βλέποντας αυτά τα δεδομένα μπορώ να φτάσω στο συμπέρασμα ότι τα train inputs και είναι αντιπροσωπευτικά και όχι ως προς τα ref inputs. Με αυτό εννοώ ότι η βασική εικόνα υπάρχει ότι ο always taken είναι τάξης μεγέθους πιο κακός αλλά σε κοντινές συγκρίσεις όπως οι παραπάνω για τον καλύτερο predictor δεν είναι καθώς ναι μεν είναι κοντά με τις προηγούμενες προβλέψεις αλλά πλέον δεν είναι καλύτερος ο Perceptron-M64-H60 αλλά ο Perceptron-M128-H32.

Όλα τα στατιστικα υπαρχουν στο repo στο εξής location advcomparch-ex1-helpcode/csvs/final4.csv

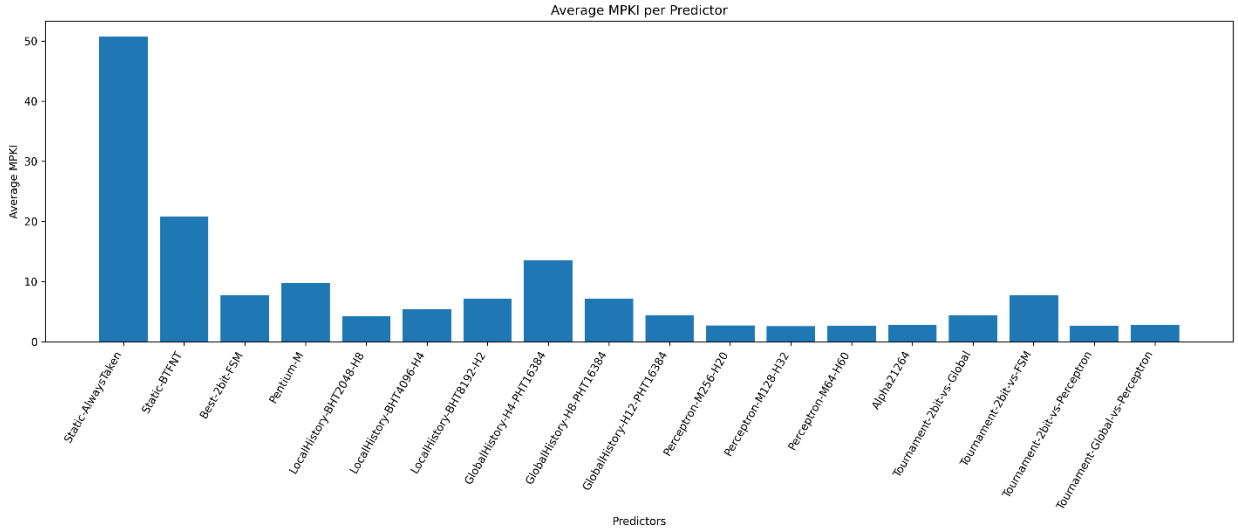
5.8 Καταλάθος extra

Στο τελευταίο ερώτημα χρειάστηκα ένα vm της aws για να το τρέξω σε ένα εύλογο χρονικό διάστημα (δεν έχω καλή ram στο laptop μου) αλλά όταν έκανα clone το repo δεν άλλαξα το cslab_branch.cpp οπότε πρακτικά έτρεξα τα ref inputs για όλους τους predictors (ναι πήρε 2 μέρες). Μιας και το έτρεξα θα παραθέσω τα αποτελέσματα και τα συμπεράσματα μου.

Benchmark	Static-AlwaysTaken MPKI	Static-BTFNT MPKI	Best-2bit-FSM (ABACBDCD:3) MPKI	Pentium-M MPKI	LocalHistory-BHT2048-H8 MPKI	LocalHistory-BHT4096-H4 MPKI
403.gcc	95.46375	53.79017	16.30222	35.88887	13.1412	16.15136
410.bwaves	15.59568	10.17474	8.366601	6.138485	2.096909	5.08002
416.gamess	30.81587	20.72657	8.606795	6.840911	2.056753	4.671536
429.mcf	128.5271	64.94077	20.7564	29.93905	14.84727	15.77088
433.milc	14.18621	13.08693	12.74798	1.235217	0.018289	0.084161
435.gromacs	25.58229	7.42789	3.023205	2.910095	2.157686	2.48788
436.cactusADM	0.059365	0.021609	0.0178	0.029675	0.013351	0.014154
450.soplex	58.1349	28.25276	7.13075	10.9993	6.337307	6.9899
459.GemsFDTD	5.263505	3.46273	0.306934	0.604226	0.263035	0.314513
470.lbm	8.691604	2.935107	0.105782	0.156756	0.041664	0.042545
483.xalancbmk	175.7598	24.42166	7.967161	12.34779	6.206265	7.986104
AVERAGE	50.73455	20.84009	7.757421	9.735489	4.289066	5.41755

Benchmark	LocalHistory-BHT8192-H2 MPKI	GlobalHistory-H4-PHT16384 MPKI	GlobalHistory-H8-PHT16384 MPKI	GlobalHistory-H12-PHT16384 MPKI	Perceptron-M256-H20 MPKI	Perceptron-M128-H32 MPKI
403.gcc	18.34417	44.24744	25.80148	13.33495	6.239068	6.54305
410.bwaves	9.5064	8.715465	3.74219	2.753978	2.201514	1.253539
416.gamess	9.866715	13.76476	6.21235	2.90832	1.7487	1.551483
429.mcf	17.09585	18.97745	14.4653	12.60454	11.4594	11.41802
433.milc	3.094982	4.265896	2.377308	0.964462	0.022559	0.022106
435.gromacs	3.645435	6.038161	4.399471	1.995273	1.504468	1.532093
436.cactusADM	0.017274	0.022164	0.015944	0.013649	0.012972	0.012801
450.soplex	7.246843	18.32803	7.2881	5.601125	4.900057	4.639467
459.GemsFDTD	0.343281	1.14852	0.314537	0.109199	0.097007	0.096013
470.lbm	0.04756	0.051909	0.041308	0.04118	0.041437	0.041563
483.xalancbmk	9.21567	34.01441	14.15457	8.774356	1.783431	1.480611
AVERAGE	7.12947	13.59766	7.164778	4.46373	2.728237	2.599159

Benchmark	Perceptron-M64-H60 MPKI	Alpha21264 MPKI	Tournament-2bit-vs-Global MPKI	Tournament-2bit-vs-FSM MPKI	Tournament-2bit-vs-Perceptron MPKI	Tournament-Global-vs-Perceptron MPKI
403.gcc	7.673041	7.365727	12.04765	15.94418	6.693194	8.426249
410.bwaves	0.568497	1.341708	2.80445	8.361184	0.995216	0.980065
416.gamess	1.487871	0.99686	3.76325	8.674934	1.469688	1.467542
429.mcf	11.47612	11.61572	13.27407	20.36917	11.8361	11.64782
433.milc	0.017644	0.011111	2.123279	12.74804	0.017827	0.017716
435.gromacs	1.615181	1.494362	2.163581	3.012191	1.679469	1.701097
436.cactusADM	0.012749	0.012972	0.014483	0.016777	0.012783	0.012808
450.soplex	4.573817	4.81239	6.202555	7.137831	4.71359	4.668408
459.GemsFDTD	0.095818	0.096884	0.277779	0.307296	0.095502	0.095553
470.lbm	0.039955	0.04115	0.041169	0.105782	0.039943	0.039338
483.xalancbmk	1.561153	2.671791	5.689564	7.979037	1.476328	1.748811
AVERAGE	2.647441	2.769153	4.400166	7.696038	2.639058	2.800491



Εν γένει παρατηρούμε το ίδιο πράγμα με το 5.7 είναι καλό για να πάρουμε μια γενική ιδέα της κατάστασης αλλά στις μικρές διαφορές δεν είναι τόσο αξιόπιστα τα train inputs όσο τα ref που είναι πιο ακριβή.

ΥΓ. όλα τα scripts, διαγράμματα, μετρικές και csvs βρίσκονται στο repo του οποίου το Link βρίσκεται στην αρχή του pdf που διαβάζεται αυτή την στιγμή (δίπλα από το σήμα του πολυτεχνείου).